

FACULTAD DE INGENIERÍA - UNLP
E1301 - E0301 Introducción a los Sistemas Lógicos y Digitales
Curso 2025 - Trabajo integrador

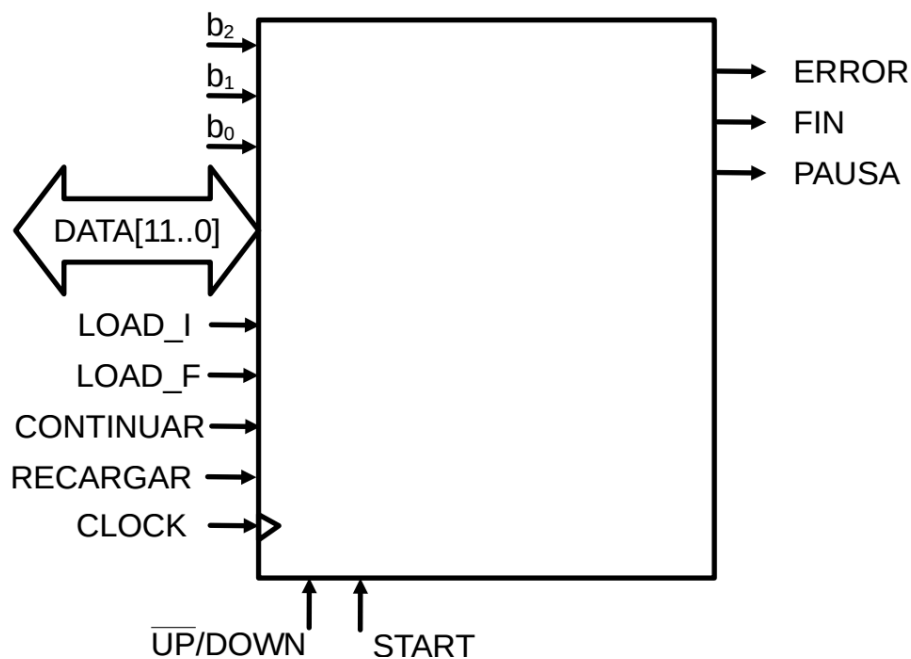
Grupo 01

Ariza, Martin (02844/4 - mar.ariza2003@gmail.com)

Pascual Velazquez, Rocio (03377/1 - rocio.pascual@alu.ing.unlp.edu.ar)

Guitera Woida, Emma (03352/2 - emmagw04@gmail.com)

Primeramente, haremos un resumen y un análisis de las variables de la máquina.



El bus de DATA permite el ingreso de los valores iniciales y finales. Los valores se inicializan en momentos distintos, determinados por las señales LOAD_I y LOAD_F.

Ambos pueden ser representados por 12 bits y asumiendo que los valores serán representados por binario sin signo, el rango de representación será [0.. 4095].

Para almacenar los valores ingresados, se utilizan 24 flip-flops tipo D, 12 para el valor inicial y 12 para el valor final.

Estos flip-flops estarán habilitados solo cuando la línea RECARGAR esté activa para evitar que el contador modifique sus valores durante la operación.

Adicionalmente utilizaremos otros 12 flip fops para almacenar las sumatorias intermedias que permitirán mostrar la información actual mediante DATA, que llamaremos Registro A

Señal	Dirección	Descripción / Función
DATA[11..0]	I/O	Bus bidireccional de 12 bits para cargar o leer el valor inicial y final, y para mostrar el valor actual.
LOAD_I	Entrada	Carga en el registro interno de valor inicial el dato presente en DATA.
LOAD_F	Entrada	Carga en el registro interno de valor final el dato presente en DATA.
b2b1b0	Entrada	Incremento/decremento programable (0–7) aplicado al contador en cada ciclo de reloj. Es un valor constante que se suma o se resta al valor inicial en DATA
PAUSA	Salida	Activa cuando b2b1b0 = 000, indicando que el contador no avanza.
$\overline{UP}/DOWN$	Entrada	Control de sentido de conteo: 0= ascendente, 1= descendente.
FIN	Salida	Pulso de 1 ciclo cuando el contador alcanza el valor final programado.
CONTINUAR	Entrada	Tras alcanzar FIN: 1 = reinicia desde el valor inicial, 0 = detiene el contador.
RECARGAR	Entrada	Habilita la carga de nuevos valores inicial/final, deteniendo el contador mientras está activa.
START	Entrada	Habilita el inicio del conteo con los valores previamente cargados.
ERROR	Salida	Se activa si los valores inicial y final son inconsistentes con el modo de operación (UP/DOWN).

FIN

Para detectar cuándo el contador alcanza el valor final (VF) y generar la salida FIN, se realiza una comparación bit a bit entre el Registro A (valor actual) y VF:

- Cada bit se compara mediante un XNOR: $XNOR = A \oplus VF$
- Luego se realiza un AND de todos los XNOR para obtener la señal de comparación actual:

Esta señal será 1 únicamente cuando $A = VF$.

Aquí surge un problema, si CONTINUAR = 0 (el conteo se detiene), el contador queda en ese valor y ACT permanece en 1 de forma indefinida.

Esto impide distinguir entre un nuevo FIN y el estado detenido en VF.

Para solucionarlo se guarda la comparación anterior en un Flip-Flop tipo D de 1 bit:

- D = ACT
- Q = COM (Comparación Anterior)
- CLK = reloj del contador

Se genera la salida FIN únicamente en el flanco en que ACT pasa de 0 a 1:

$$FIN = ACT \cdot \overline{COM}$$

CARGA DE REGISTROS

La carga de los registros que almacenan los valores inicial (VI) y final (VF) solo se realizará cuando RECARGAR = 1.

Para diferenciar cuál de los dos registros se carga a través del bus DATA[11..0], se utilizan las señales LOAD_I y LOAD_F.

Cada registro se implementará con 12 flip-flops tipo D con entrada de Enable.

El Enable de cada registro será:

$$EVI = RECARGAR \cdot LOAD_I$$

$$EVF = RECARGAR \cdot LOAD_F$$

ERROR

La salida ERROR será una flag informativa debería activarse en los siguientes casos:

- Si $VF < VI$ y $\overline{UP/DOWN} = 0$
- Si $VF > VI$ y $\overline{UP/DOWN} = 1$

Para la comparación de valores utilizaremos un sumador Ripple Carry de 12 bits, conectando 12 sumadores completos. Como la señal ERROR se evalúa únicamente en la carga inicial y no en cada ciclo de conteo, la implementación es suficiente.

Restamos el vi con el vf, utilizando la suma del primero con el complemento a 2 del segundo, obteniendo la siguiente fórmula:

$$R = VI - VF = VI + \overline{VF} + 1$$

Luego analizaremos el bit del carry out, para determinar las magnitudes:

- Si C_{out} es 1, $VF \leq VI$
- Si C_{out} es 0, $VF > VI$

Con el valor del bit carry out y el valor de entrada de $\overline{UP/DOWN}$, planteamos:

C_{out}	$\overline{UP/DOWN}$	ERROR
1	1	0
1	0	1
0	1	1
0	0	0

Del cual obtenemos la expresión final de ERROR siendo:

$$ERROR = C_{out} \cdot \overline{\overline{UP/DOWN}} + \overline{C_{out}} \cdot \overline{UP/DOWN} = C_{out} \oplus \overline{UP/DOWN}$$

PAUSA

Para la salida PAUSA, tendremos que verificar que $b_2b_1b_0$ es 0. Por ende, usaremos una compuerta NOR conectada a tierra, de manera que obtendremos una salida 1 cuando los tres bits sean 0

$$PAUSA = \overline{b_2 + b_1 + b_0}$$

Funcionamiento Principal

El funcionamiento del contador se puede describir con la siguiente función:

$$A_{\text{nuevo}} = \begin{cases} A + K & \text{si } \overline{UP/DOWN} = 0 \\ A - K & \text{si } \overline{UP/DOWN} = 1 \end{cases} \quad \text{con } K=b_2b_1b_0$$

Para cargar en los flip-flops intermedios (a los que llamaremos Registro A) el valor inicial o el valor intermedio del conteo, utilizaremos un multiplexor de 2 entradas:

- Entrada 0: $A \pm K$
- Entrada 1: Valor inicial del contador

El selector del multiplexor será una señal lógica que nos permita distinguir. Cargaremos el valor inicial en el Registro A en 2 caso donde:

- Se carga por primera vez $\rightarrow \text{RECARGAR}=1$
- Se llegó al resultado y se reinicia la cuenta $\rightarrow \text{FIN}=\text{CONTINUAR}=\text{START}=1$

Por ende la ecuación quedaria

$$SEL = \text{RECARGAR} + (\text{FIN} \cdot \text{CONTINUAR} \cdot \text{START})$$

Este Multiplexor repetirá con cada bit.

El Registro A se actualiza únicamente cuando:

- Se realiza la carga inicial ($\text{RECARGAR} = 1$)
- El contador está activo y no en pausa ($\text{START} = 1$ y $\text{PAUSA} = 0$)

Por lo tanto, la señal de Enable de los FF del registro será:

$$EN_a = \text{RECARGAR} + (\text{START} \cdot \overline{\text{PAUSA}})$$

Durante la sumatoria debemos mostrar en DATA las sumas intermedias, pero debemos evitar que interfiera con el ingreso de los valores durante la carga. Para esto usaremos tristate por cada bit, en el cual conectaremos cada bit de dato y la señal que me indique que estamos en la suma. La salida estará conectada a DATA, siendo el valor real si la señal es 1 o 0 si la señal es 0. Lo haremos con cada bit del registro.

La señal tendrá esta lógica:

$$EN_s = \text{START} \cdot \overline{\text{PAUSA}} \cdot \overline{\text{FIN}}$$

Para realizar la sumatoria que permite el avance del contador, deberemos utilizar una lógica similar al utilizado en el ERROR. Para esto utilizaremos un multiplexor de 2 entradas:

- Entrada 0: K

- Entrada 1: $\overline{K} + 1$

El selector será la señal $\overline{UP/DOWN}$.

El multiplexor se repetirá con cada bit.

La salida del multiplexor pasará a un sumador de 12 bits implementado con sumadores completos en cascada (ripple carry), el cual recibe también el valor actual del Registro A.

En caso de requerirse mayor velocidad, el diseño podría optimizarse empleando un sumador con acarreo anticipado (carry look-ahead. El sumador de 12 bits es combinacional y siempre calcula la próxima suma/resta.

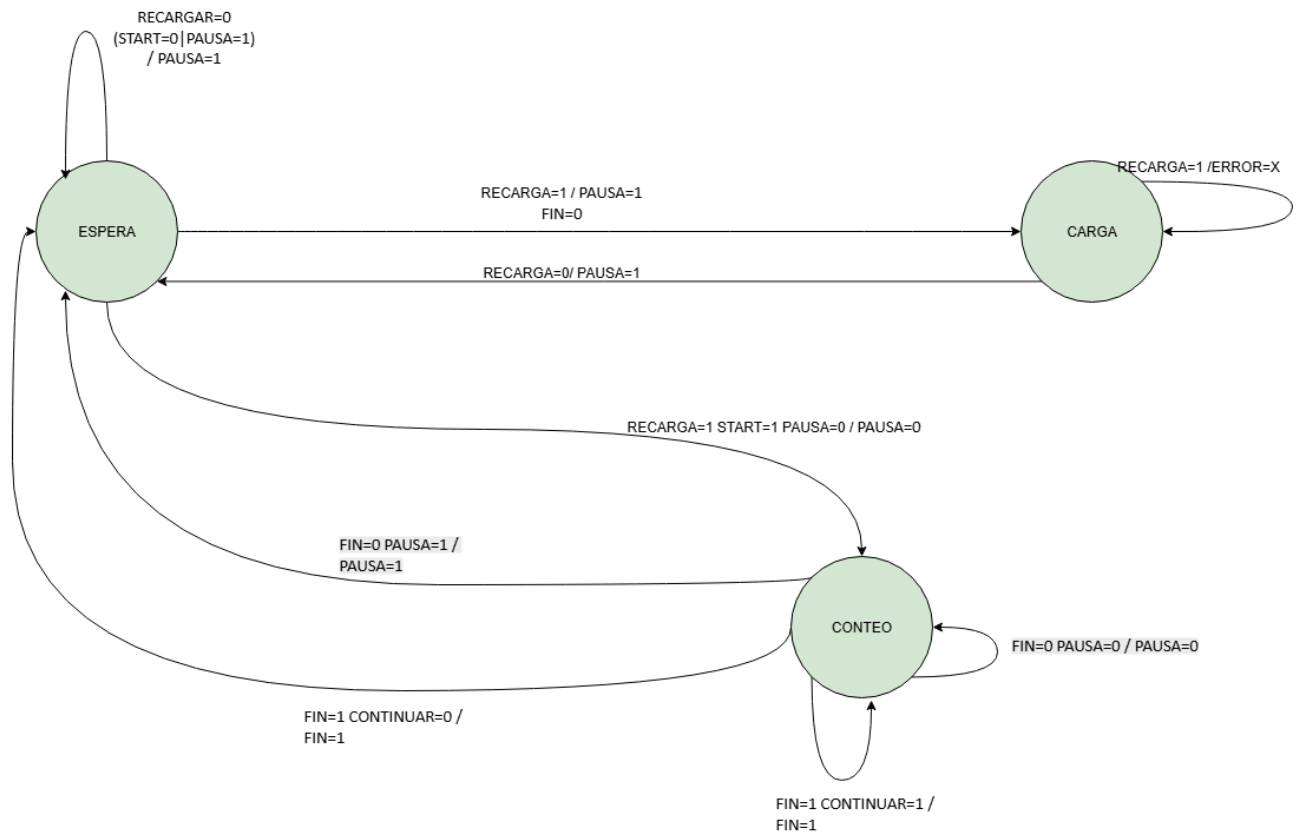
El avance del contador se controla mediante la señal EN_a que habilita la carga del Registro A, evitando que el valor cambie mientras el contador está detenido o en pausa.

La máquina oscila entre 3 posibles estados, según las entradas y la cuenta actual del contador:

- ESPERA: dónde START=0, RECARGAR=0 y CONTINUAR=0, se da cuando b2b1b0=000 o el contador está detenido
- CARGA: donde RECARGAR=1, permite el ingreso de datos por la salida bidireccional y se confirma su transferencia mediante LOAD_I y LOAD_F. También se chequea el error
- CONTEO: Cuando PAUSA = 0, FIN = 0, START = 1 y RECARGAR = 0, el contador está activo, sumando o restando según UP/DOWN, reflejando su valor en DATA[11..0]. Si alcanza VF, genera un pulso en FIN: si CONTINUAR = 1 reinicia desde VI, y si CONTINUAR = 0 pasa a ESPERA. Si b2b1b0 = 0 o RECARGAR = 1, el contador se detiene y sale de este estado

Estado Actual	Entradas	Próximo Estado	Salidas
Espera	RECARGAR=1	Carga	PAUSA=1 FIN=0
	RECARGAR=0 & START=1 & PAUSA=0	Conteo	PAUSA=0
	RECARGAR=0 & (START=0 PAUSA=1)	Espera	PAUSA=1
Carga	RECARGAR=1	Carga	ERROR=X
	RECARGAR=0	Espera	PAUSA=1
Conteo	FIN=1 & CONTINUAR=1	Conteo	FIN=1
	FIN=0 & PAUSA=0	Conteo	PAUSA=0
	FIN=0 & PAUSA=1	Espera	PAUSA=1
	FIN=1 & CONTINUAR=0	Espera	FIN=1

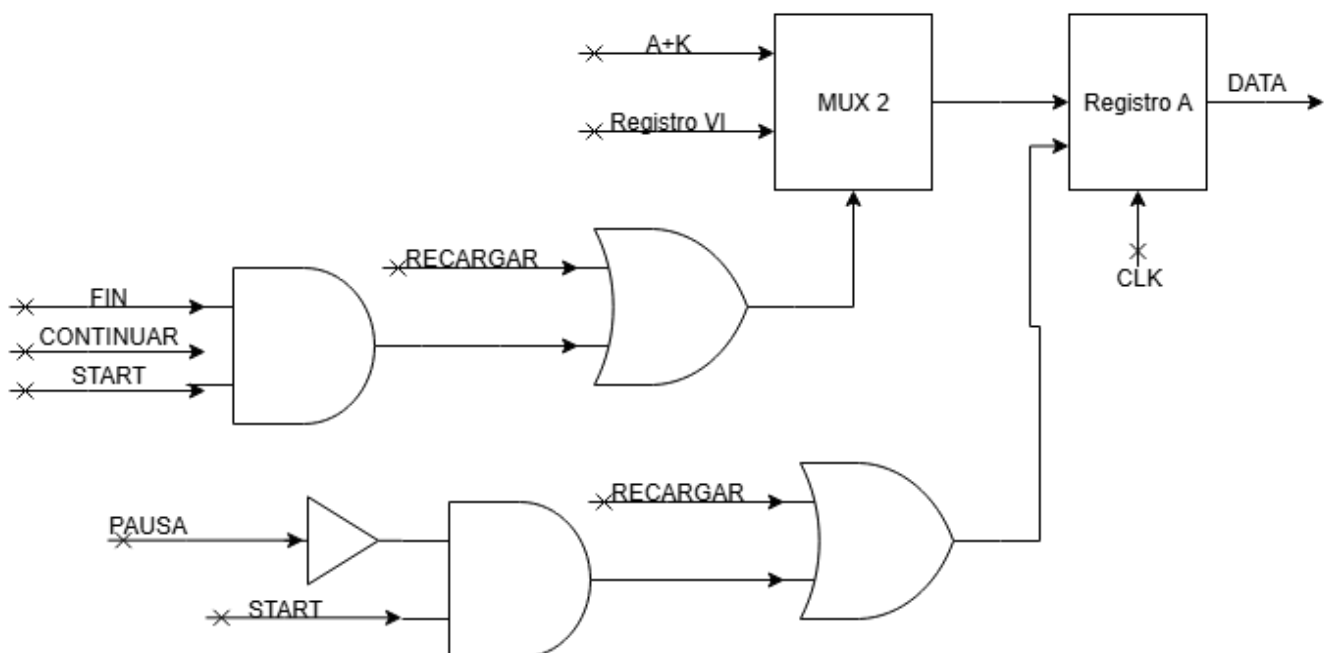
El diagrama de estado correspondiente sería:



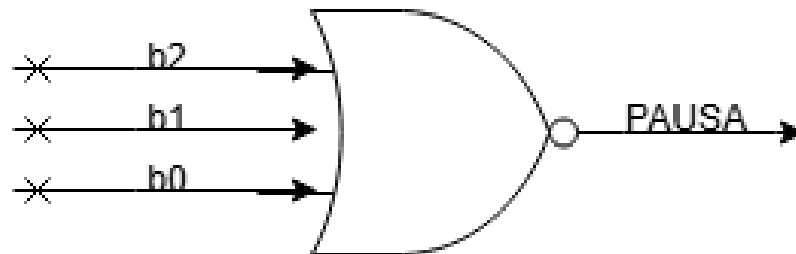
Esquemas del circuito

Hemos graficado los bloques principales del circuito:

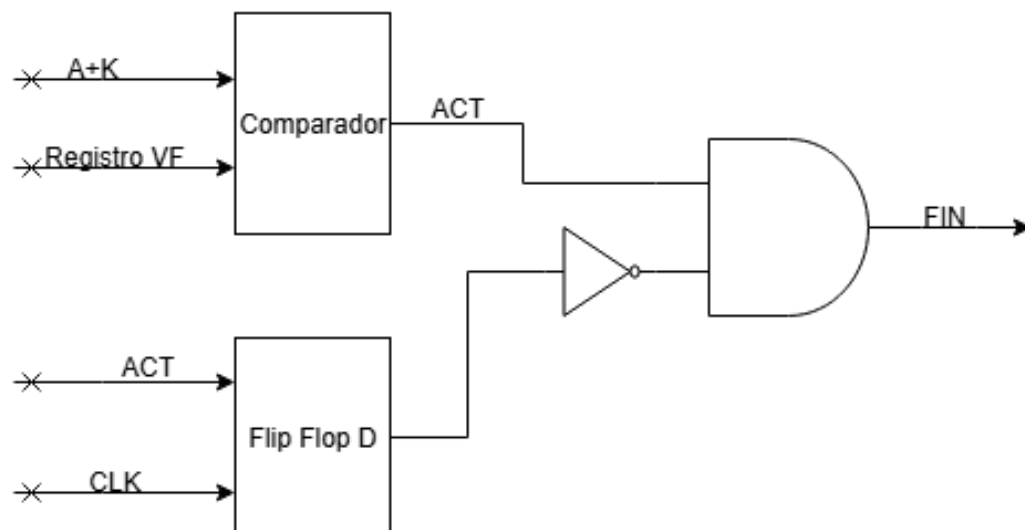
Carga del Registro A



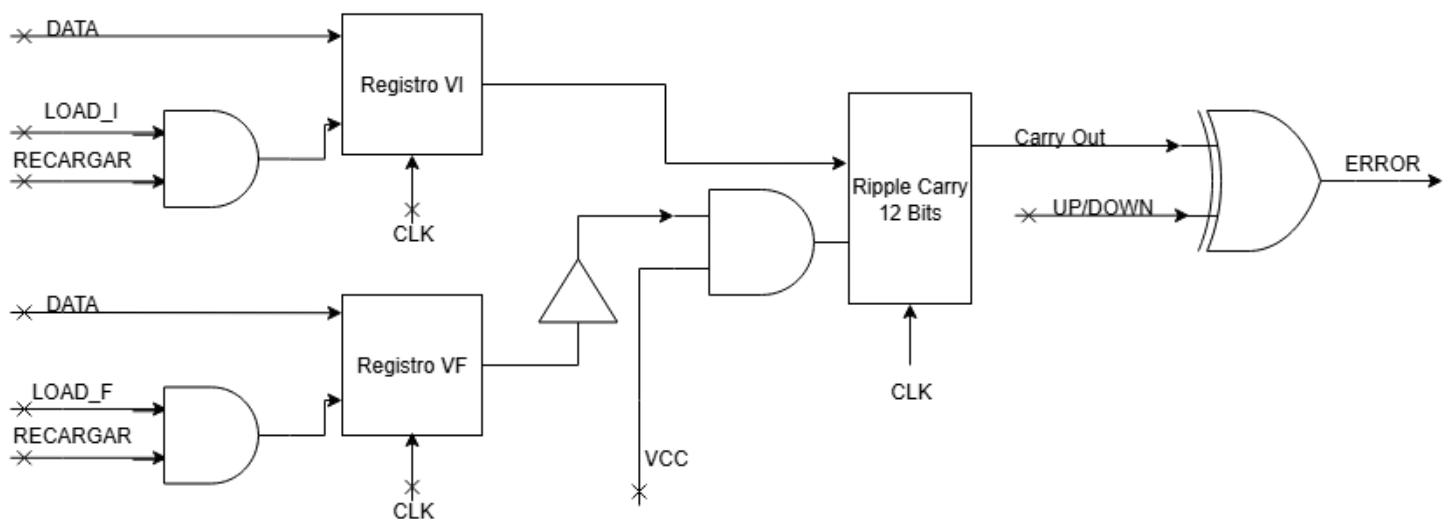
Pausa



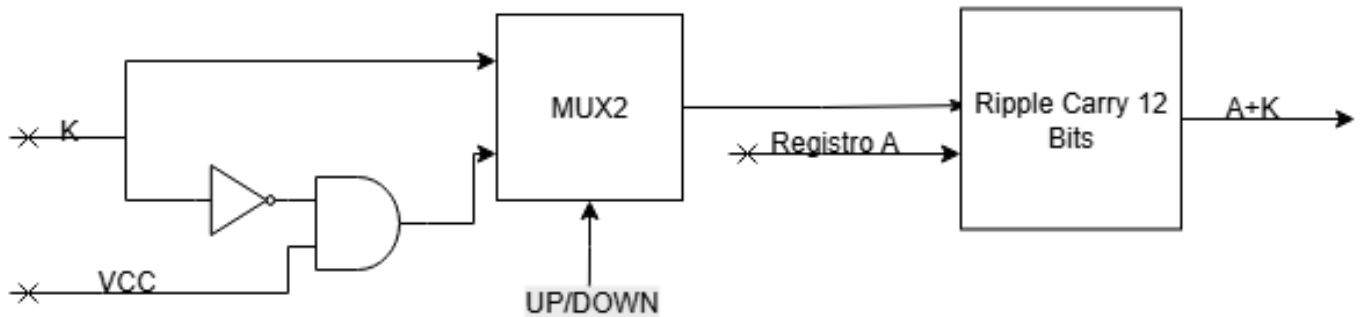
Fin



Carga de registros y Error



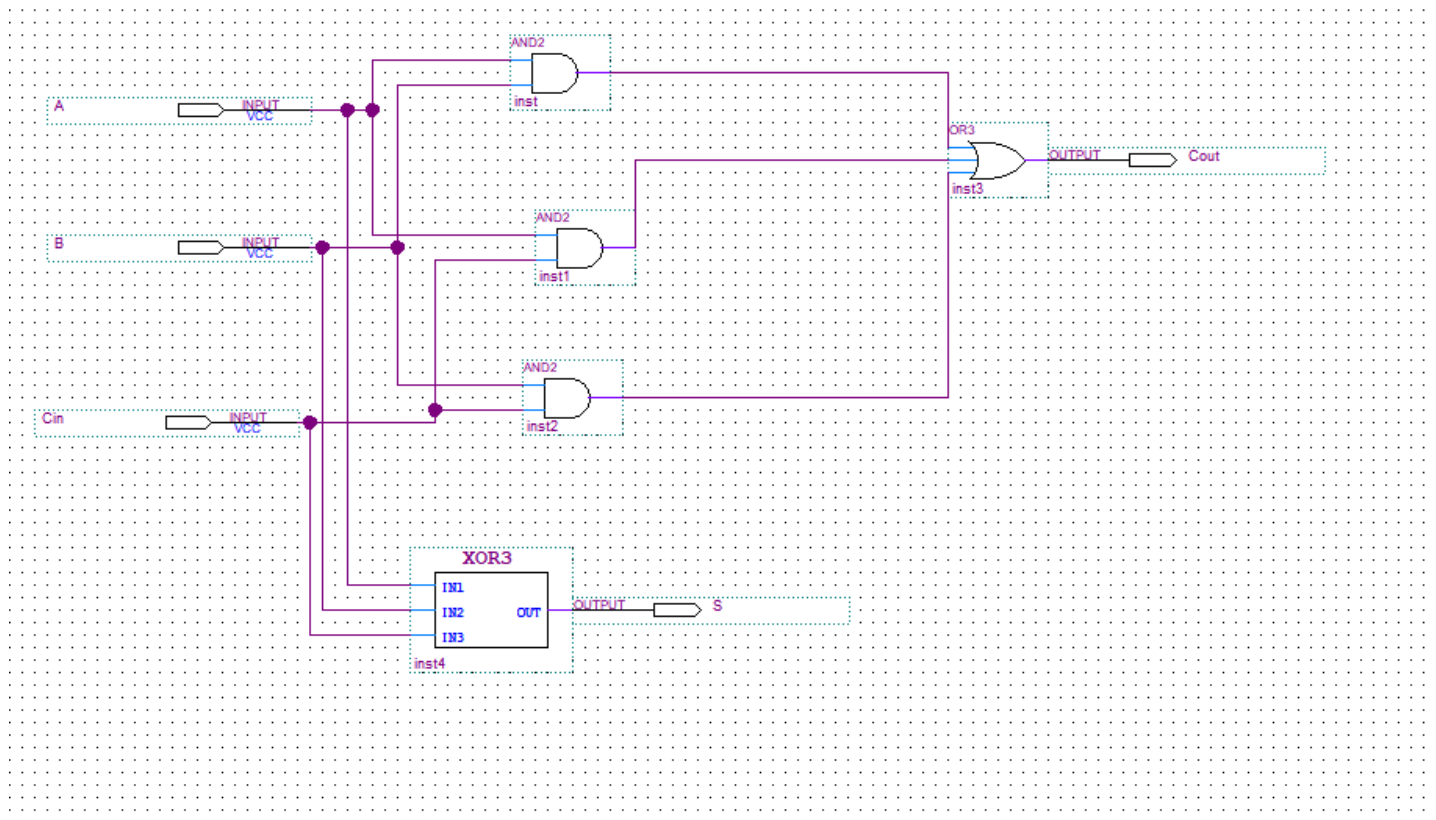
Sumador



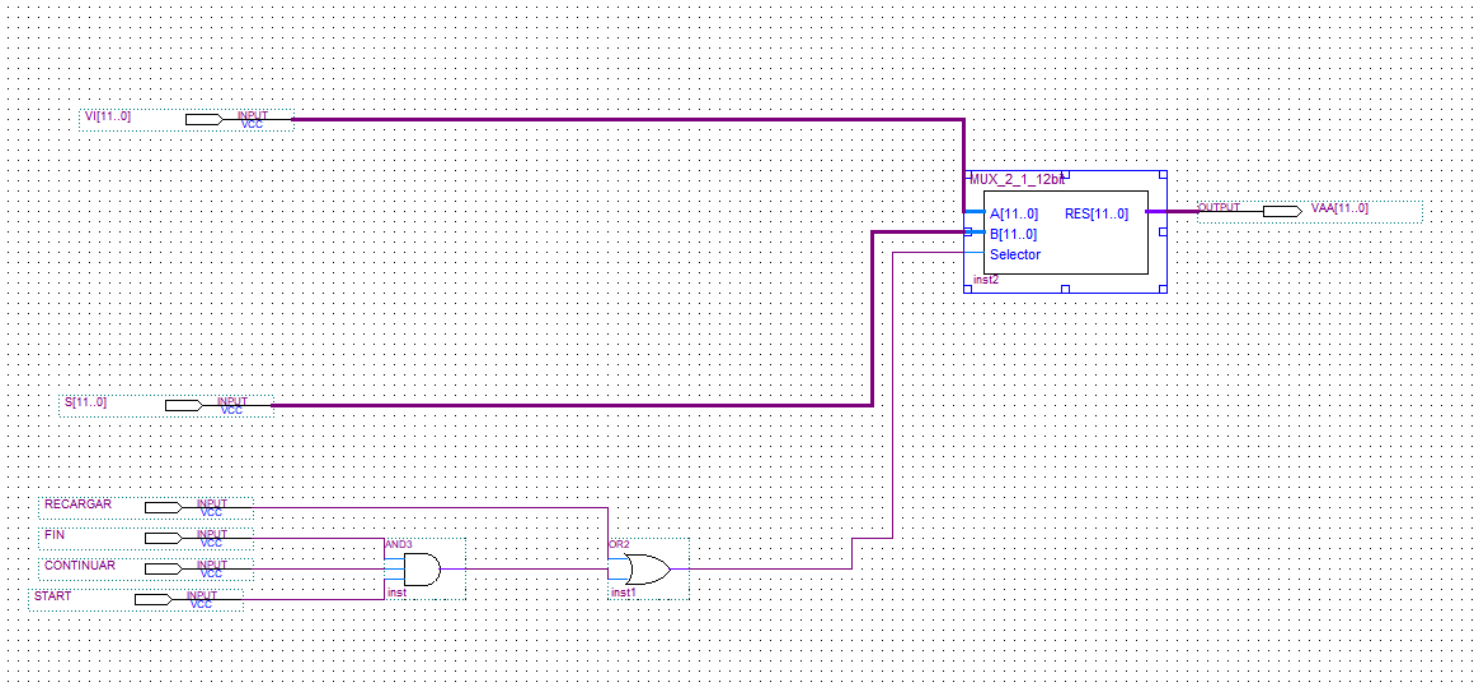
IMPLEMENTACIÓN EN QUARTUS II

Para la implementación en Quartus se crearon los siguientes Symbols:

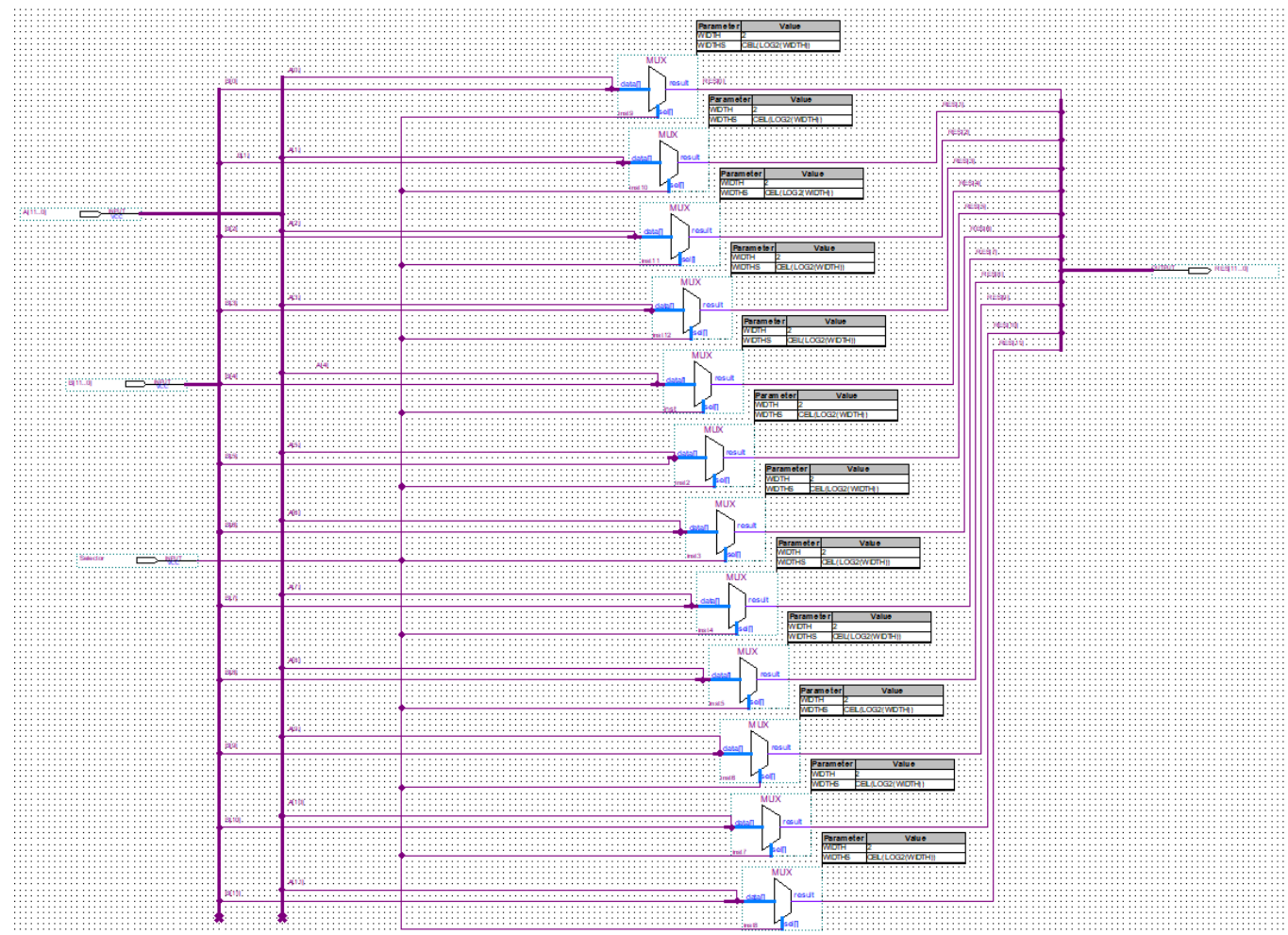
- SumadorCompleto_1bit: sumador completo basado en la teoría



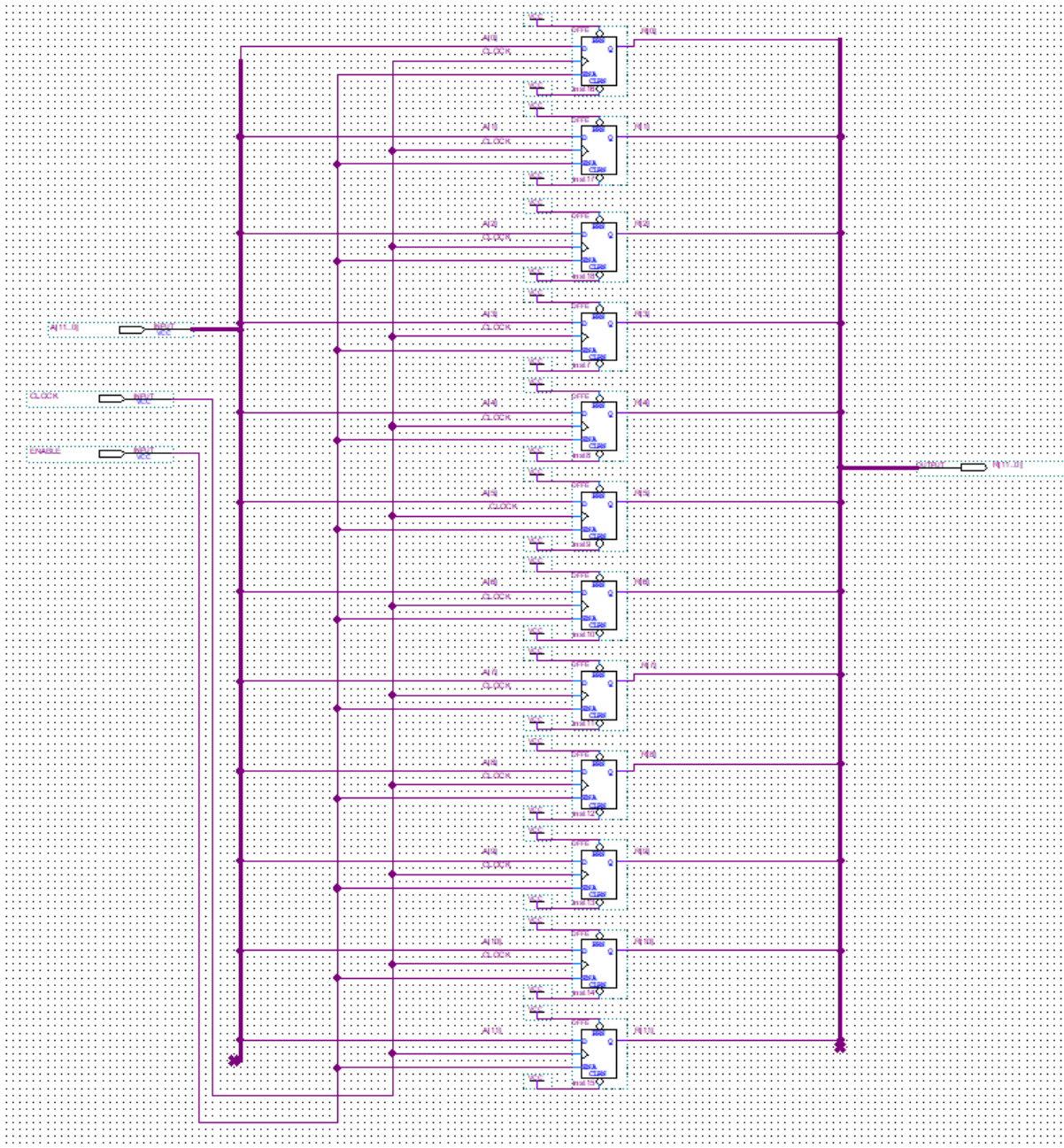
- CARGA DE VALORES: basado en el esquema Carga de registros y error, utilizamos el symbol del sumador para realizar el Ripple Carry
- CARGA A: basada en el esquema Carga del Registro A



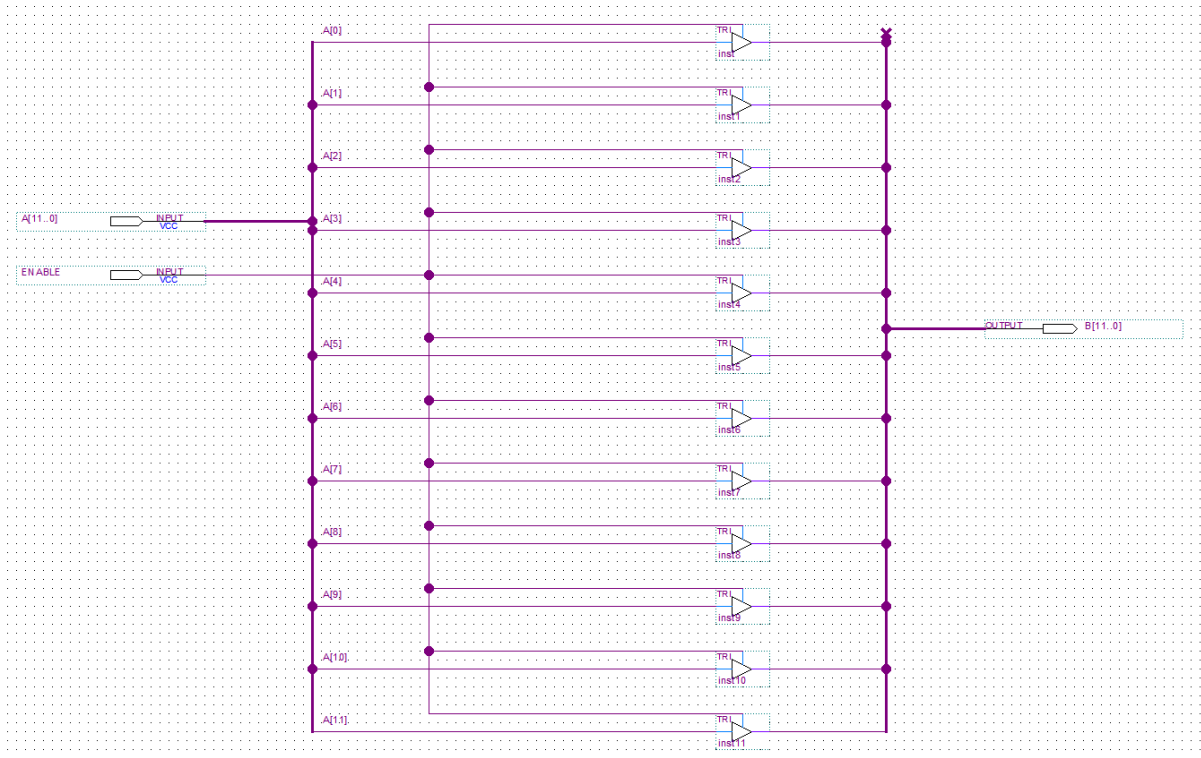
- MUX_2_1_12bit: la implementación de un multiplexor 2:1 para 12 bits



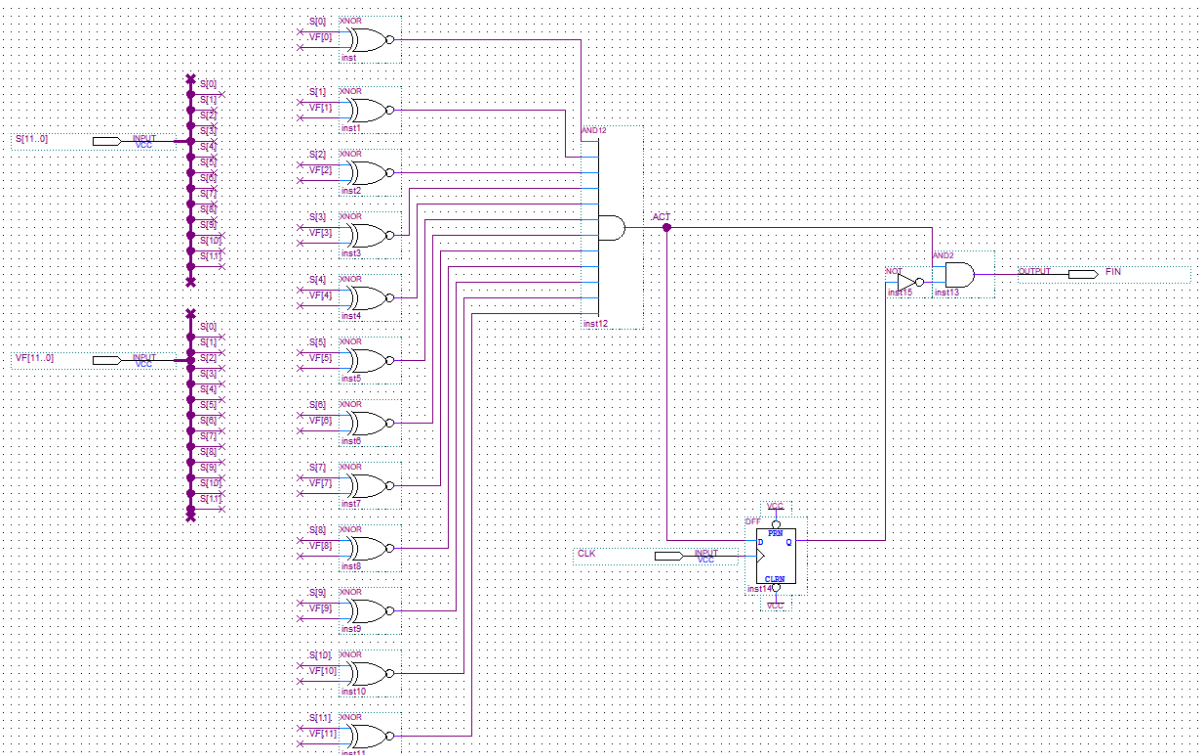
- REGISTRO: implementación de un registro de 12 bits mediante Flip Flops de 2 bits con enable



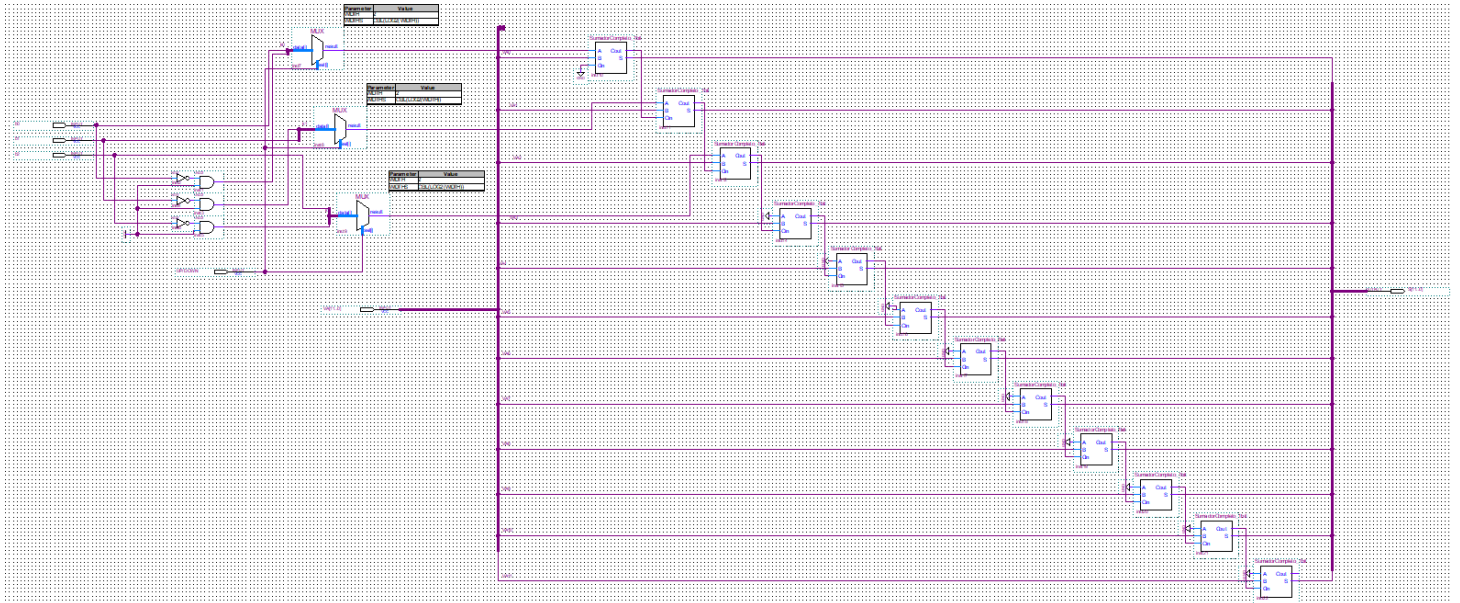
- TRI_12: permite manejar un bus de 12 bits mediante buffers triestado controlados por una única señal de habilitación. Internamente, cada uno de los 12 bits de datos atraviesa un buffer TRI individual,



- FLAGFIN: basado en el diagrama FIN



- SUMADOR: basado en el esquema Sumador



Máquina final

